Rust suite: GF(2^n) inversion chain, trinomial fold, shared field tables, IFMA rows

Date: 2026-11-04. **Repository:** `aburan28/cryptanalysis`, production Rust crate `suite/`; study `experiments/suite-gf2-inv-chain-20261104/`. **Class:** engineering; every perfindex fingerprint is identical to `main`'s (gate valid). **Baseline:** `origin/main` 304ab63c7. **Instruments:** Callgrind instructions (scalar paths only: Valgrind does not emulate AVX-512) and the native paired wall comparison (AVX-512 kernels on; 7 rounds; L0, CPU 3). **Outcome:** performance index **1.215×** (95% CI 1.176–1.285). Single inversion 4.14×, `KoblitzCurve::mul` 1.46×, signed Frobenius rho at n = 41 1.20×, the k0n31 IC rung 1.11×, PDP enumeration 1.72–1.93×.

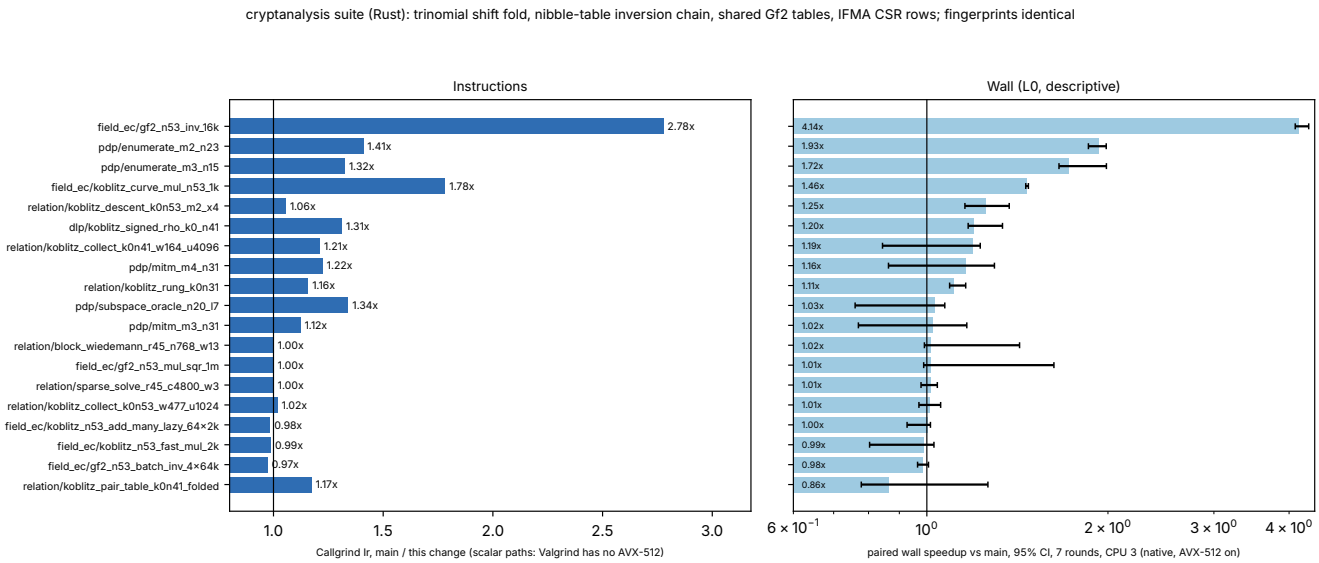


Figure 1: Instruction ratios (main / change) and paired wall speedups against `main` (`data/measurements.json`).

What changed

Inversion chain by nibble tables. Itoh–Tsujii raises to `2^k` for each run length `k` of the chain (1, 3, 6, 13, 26 at n = 53). The map `a -> a^(2^k)` is F2-linear, so a run with `k >= 2` is now one lookup per nibble (`ceil(n/4)` independent loads) instead of `k` dependent squarings: inversion at n = 53 is 4.14× faster and runs 2.78× fewer instructions.

Trinomial shift fold. For `z^n + z^t + 1`, a product folds by two rounds of `h ^ (h << t)` instead of `ceil((n-1)/8)` table lookups. This covers 33 of the 62 sizes, including n = 31 and n = 41. Pentanomials keep the table, because a four-term shift fold measured 0.84× at n = 53.

Shared tables. The reduction and chain tables are built once per modulus. `KoblitzCurve::mul`, which builds a `Gf2` on every call, no longer rebuilds 16 KiB of tables each time (1.46×).

AVX-512 IFMA CSR rows in the sparse linear algebra. They are chosen once per product. A same-binary comparison measured 30.0–30.6 ms against 32.1–36.2 ms. Against `main` here they are neutral within noise.

Not improved

The n = 53 multiply, batch inversion, lazy addition and ladder are unchanged natively (0.98–1.02×). Under Callgrind they run 1–3% more instructions, from the trinomial branch. `koblitz_pair_table_k0n41_folded` reads 0.86× natively, but its interval (0.78–1.26) contains 1, it runs 1.17× fewer instructions, and it measured 1.018× against the pre-change head; it counts as neither a regression nor a gain. These are kernel speedups, not a complete-solve row: no matched-rho comparison is claimed. The next lever on the k0n53 descent is the batched slope inversion, about 20% of its scalar instructions.